

Memo

1. Problem and stakes

Lending Club grades every loan application on a scale from A to G. Grades A, B, and C are considered prime - these borrowers get lower interest rates because they are seen as low risk. Grades D through G are higher-risk, so those borrowers pay more. Our task was to build a model that can predict which bucket a borrower will fall into using only the information available at the time they apply - variables like income, credit history, debt load, and loan purpose. We were not allowed to use any additional information that would come after.

If we miss a risky borrower and fund their loan, Lending Club could lose the entire principal if that loan defaults. If we incorrectly flag a good borrower as risky and decline them, we just lose the interest we would have earned. The scenario tells us that funding a bad loan costs roughly 10 times more than turning away a good one. About 18.5% of loans in our dataset are higher-risk, so there is real risk - catching even a fraction more of those bad loans early makes a meaningful difference.

A good second-opinion model gives the risk team a fast automated signal to catch risky applications before any money goes out the door, rather than finding out after the fact.

2. Recommendation

Our recommendation is to deploy the Logistic Regression model.

We built and compared four models - Logistic Regression, Decision Tree, Neural Network, and Random Forest - all trained on the exact same data pipeline to ensure equal comparison. After looking at both training and test performance, Logistic Regression came out as the most reliable choice:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8240	0.5634	0.2162	0.3125	0.7711
Decision Tree	0.8070	0.4403	0.1595	0.2341	0.7219
Neural Network	0.7835	0.3994	0.3378	0.3660	0.7112
Random Forest	0.8270	0.6765	0.1243	0.2100	0.7877

Here is why we went with the Logistic Regression over the others:

- It has strong ROC-AUC performance, meaning it is effective at ranking risky borrowers above prime ones. That is the most important metric here because it measures the model's overall discrimination ability, not just how it performs at one specific cutoff.
- With only 18.5% of loans being risky, the dataset is imbalanced. A model that just predicts "prime" for everyone would score 81.5% accuracy while being completely useless. Logistic Regression had the best Test F1 (0.3125) and the best Test Recall (0.2162) of all four models, meaning it actually catches more risky loans on unseen data than any of the others.
- The most important reason is generalization. Both the Random Forest and the Neural Network scored near-perfectly on the training set but dropped significantly on the test set - the Random Forest went from a Train F1 of 1.00 down to a Test F1 of 0.21, a gap of 0.79. That means it memorized the training data without learning anything that actually generalizes. Logistic Regression has a F1 gap of only 0.04 and an AUC gap of only 0.036, by far the smallest of any model. A model that generalizes reliably is always more trustworthy in production.

3. Top 5 drivers

The table below shows the five features the Logistic Regression relied on most heavily, measured by the absolute size of each feature's scaled coefficient. A larger coefficient means the model weighted that feature more when deciding between risky and prime. All five are available at application time.

#	Feature	Business Interpretation
1	<i>term</i>	Loan term is the single strongest signal in the model - borrowers requesting 60-month loans are more likely to be graded higher-risk than those on 36-month terms. Lending Club reserves shorter terms for its most creditworthy applicants, so the term essentially reflects the lender's own confidence in the borrower.
2	<i>verified_income_Verified</i>	Surprisingly, having income verified is associated with higher-risk grades. This likely happens because Lending Club requires verification precisely when an application already looks shaky - so the flag captures pre-existing suspicion rather than causing risk. It is a good example of a feature that means something different in context than it does at face value.
3	<i>total_debit_limit</i>	A higher total debit limit means other financial institutions have already extended significant credit to this borrower, which signals established creditworthiness. Borrowers with low debit

		limits tend to have thinner or weaker credit profiles, which pushes them toward higher-risk grades.
4	<i>num_open_cc_accounts</i>	The number of open credit card accounts reflects how actively a borrower is using credit. Too many open accounts can signal financial overextension, while too few may point to a thin credit history - both of which Lending Club's model associates with higher risk.
5	<i>credit_utilization_rate</i>	Borrowers who are using up a high share of their available credit are already financially stretched, leaving little buffer to absorb new loan payments. We built this feature ourselves from the raw data.

4. Tradeoffs and threshold

The default threshold of 0.5 assumes that missing a risky borrower and wrongly declining a good one are equally bad mistakes. For this problem, that is not true at all.

The cost asymmetry in this scenario is clear: funding a bad loan costs about 10 times more than declining a good one. In terms of the confusion matrix, that means:

- False negative (we miss a risky borrower, they get funded) = 10 cost units
- False positive (we flag a good borrower as risky, they get declined) = 1 cost unit

When you use a threshold of 0.5, the model only flags someone as risky if it is more than 50% sure. That means a lot of risky borrowers slip through because the model is not confident enough to flag them. And given that each one of those slip-throughs costs 10 units, you end up paying a lot for that caution.

We lowered the threshold to 0.10. At this level, the model flags anyone it assigns at least a 10% probability of being higher-risk. That means more false-positives - good borrowers that get incorrectly flagged - but the false negatives drop significantly, and under a 10:1 cost ratio, that trade-off is clearly worth it.

To actually confirm this rather than just assume it, we ran a cost sweep across thresholds from 0.10 to 0.70. At each step we calculated total cost as (false positives * 1) + (false negatives * 10). The threshold at 0.10 came out with the lowest total cost - roughly 40% of what it would be at 0.5. The downside is real but manageable: precision drops, meaning not every application we flag will actually be risky. But we would rather the risk team spend time reviewing some good applications than have bad loans get funded without anyone taking a second look. A false positive costs the reviewer some time. A false negative costs Lending Club principal.

5. Operational recommendation

What we would suggest: use the Logistic Regression at threshold 0.10 as an automated pre-screening step before any loan officer reviews an application. Any application where the model's predicted probability of being higher-risk exceeds 0.10 gets automatically flagged for manual review by a senior underwriter. Everything below the threshold moves forward through the normal process. This keeps the workload manageable - the model handles the initial pass, and employees focus their attention on the applications the model is unsure about.

One important caveat: this model was trained on data from early 2018 and will go stale. Borrower behavior shifts with the economy - interest rates rise, unemployment changes, the mix of applicants changes. If the model keeps running on 2018 patterns while the world moves on, its predictions will drift and the threshold of 0.10 will no longer be calibrated correctly. We would recommend retraining on fresh data at least every quarter and monitoring whether the model's predictions are still aligned with Lending Club's actual grade assignments. If the gap starts widening, that is the signal to retrain before trusting the model's output for business decisions.